CHAPTER 2 - Introduction to Spring Core

2.1 Objectives

- To introduce the Spring Core Component
- To know about the Inversion of Control (IoC) and Dependency Injection concepts
- To understand the BeanFactory container and ApplicationContext container of the Spring Core

2.2 Audience

This document is designed for Java programmers with a need to understand the Spring framework in detail specifically the Spring Core container and its components. The trainees who have understood the features of the Spring Framework with an overview of the components in it are the intended audience for this material.

2.3 Prerequisites

Before proceeding with this document you should have a fair knowledge about the Spring Architecture.

2.4 Course Content

2.4.1 Background to using Containers

A common issue that system architects face while building the enterprise application is how to wire together different elements: how do you fit together this web controller architecture with that database interface backing when they were built by different teams with little knowledge of each other. A number of frameworks have taken a stab at this problem, and several are branching out to provide a general capability to assemble components from different layers. These are often referred to as lightweight containers, examples include Pico Container, and Spring. Underlying these containers are a number of interesting design principles, things that go beyond both these specific containers and indeed the Java platform.

2.4.2 Components and Services

Components mean a glob of software that's intended to be used, without change, by an application that is out of the control of the writers of the component. By 'without change', we mean that the using application doesn't change the source code of the components, although they may alter the component's behavior by extending it in ways allowed by the component writers.

A service is similar to a component in that it's used by foreign applications. The main difference is that components are used locally (think jar file, assembly, dll, or a source import). A service will be used remotely through some remote interface, either synchronous or asynchronous (Examples: Web Services, messaging system, RPC, or socket.)

2.4.3 Tight coupling

Before even we get in to abbreviation of IOC and DIP, let’s first understand the problem faced in the current application development. Consider an example where we have a customer class which contains an address class object.

```
package com.tcs.ilp.sping.examples;

public class Customer {

    private String custName = null;
```

```
    private Address custAddress = new Address ();

    ….

}
```

The biggest issue with the code is tight coupling between classes. In other words the customer class depends on the address object. So for any reason address class changes it will lead to change and compiling of 'Customer' class also.

So let's put down problems with this approach:

- ✓ The biggest problem is that Customer class controls the creation of Address object.
- ✓ Address class is directly referenced in the Customer class which leads to tight coupling between Address and Customer objects.
- ✓ Customer class is aware of the Address class type. So if we add new Address types like home address, office address it will lead to changes in the Customer class also as customer class is exposed to the actual Address implementation.
- ✓ So if for any reason the Address object is not able to create the whole customer class will fail in the constructor initialization itself.

2.4.4 Inversion of Control (IoC)

The term Inversion of Control (IoC) refers to a programming style where a framework or runtime, controls the program flow. Inversion of control means we are changing the control from normal way. It works on Dependency Inversion Principle.

A software architecture with this design inverts control as compared to traditional procedural programming: in traditional programming, the custom code that expresses the purpose of the program calls into reusable libraries to take care of generic tasks, but with inversion of control, it is the reusable code that calls into the custom, or problem-specific, code.

Inversion of control is used to increase modularity of the program and make it extensible

In object-oriented programming, there are several basic techniques to implement inversion of control. These are:

- Using a factory pattern
- Using a service locator pattern
- Using a dependency injection
- Using a contextualized lookup
- Using Template method design pattern
- Using strategy design pattern

In an original article by Martin Fowler, the first three different techniques are discussed. In a description about inversion of control types, the last one is mentioned. Often the contextualized lookup will be accomplished using a service locator.

More important than the applied technique, however, is the optimization of the purposes.

2.4.5 Dependency Inversion Principle (DIP)

The Dependency Inversion Principle as defined by Martin Fowler states that:

1.High level modules should not depend upon low level modules. Both should depend upon abstractions.

2.Abstractions should not depend upon details. Details should depend upon abstractions.

The Dependency Inversion principle (DIP) helps us to develop loosely couple code by ensuring that high-level modules depend on abstractions rather than concrete implementations of lower-level modules. The Inversion of Control pattern is an implementation of this principle

2.4.6 Spring IOC – Dependency Injection

Consider a scenario of child trying to fetch things from a refrigerator: when the child goes and gets things out of the refrigerator by itself, the child can cause problems. The child might leave the door open, and might get something that Mommy or Daddy doesn't want the child to have. It might even be looking for something we don't even have or which has expired. What the child should be doing is stating a need, "I need something to drink with lunch," and then the associated members will make sure that it has something when the child sits down to eat.

Spring Uses “IoC’s Dependency Injection (DI) Pattern” to implement all its own features/ modules and all application. DI was coined by Martin Fowler and the definition is as given below:

Passing the service to the client, rather than allowing a client to build or find the service, is the fundamental requirement of the pattern. “Don’t Call me, I will call you !!!“ –way - the framework calls your code, not the reverse.

The various way to Inject/Pass the Dependent objects in DI Pattern in Spring:

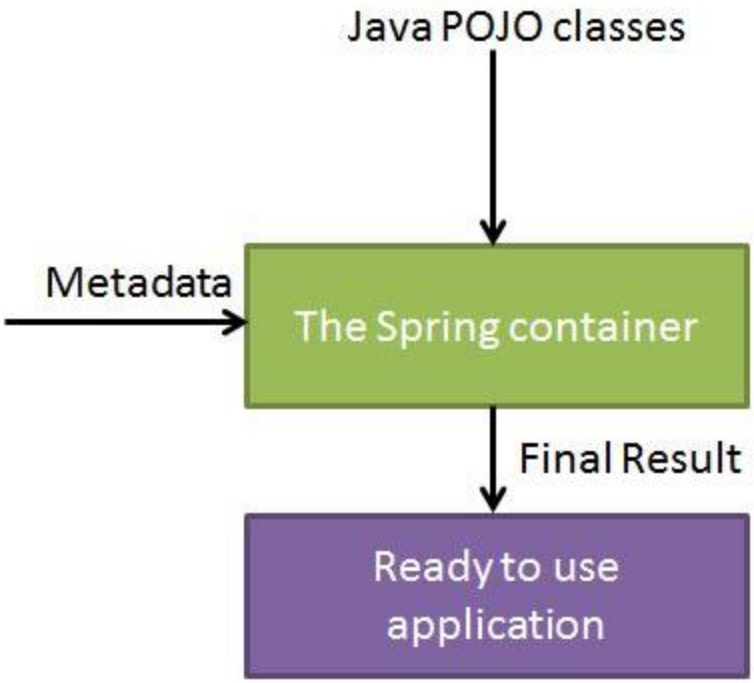
- Constructor Injection - the dependencies are provided through a class constructor
- Setter Injection - client exposes a setter method that the injector uses to inject the dependency

DI concepts include the following type of injection as well, but not frequently used in Spring :

- Interface Injection
- Parameter Injection

2.4.7 Spring IOC Containers

The Spring container is at the core of the Spring Framework. The container will create the objects, wire them together, configure them, and manage their complete life cycle from creation till destruction. The Spring container uses dependency injection (DI) to manage the components that



make up an application. These objects are called Spring Beans which we will discuss in next chapter.

The container gets its instructions on what objects to instantiate, configure, and assemble by reading configuration meta-data provided. The configuration meta-data can be represented either by XML, Java annotations, or Java code.

Figure 1: How Spring Container works

The above diagram is a high-level view of how Spring works. The Spring IoC container makes use of Java's Plain Old Java Objects (POJO) classes and configuration meta-data to produce a fully configured and executable system or application. Spring provides following two distinct types of containers.

2.4.7.1 Spring BeanFactory Container

This is the simplest container providing basic support for DI and defined by the ***org.springframework.beans.factory.BeanFactory*** interface. The BeanFactory and related interfaces, such as ***BeanFactoryAware***, ***InitializingBean***, ***DisposableBean***, are still present in Spring for the purposes of backward compatibility with the large number of third-party frameworks that integrate with Spring.

2.4.7.2 Spring ApplicationContext Container

This container adds more enterprise-specific functionality such as the ability to resolve textual messages from a properties file and the ability to publish application events to interested event listeners. This container is defined by the org.springframework.context.ApplicationContext interface.

2.4.8 Spring Bean Definitions

The Objects/POJOs of the applications, when managed by IoC Container are called 'Beans' with complete life cycle as below :

- Instantiated when needed
- Assembled as required
- Destroyed when no longer needed

The bean definitions contain the following :

- How to create a beans
- Beans life cycle details
- Beans dependencies

There are many properties that make up the bean definition. The properties of the bean definition are :

- **class**: This attribute is mandatory and specify the bean class to be used to create the bean.
- **name**: This attribute specifies the bean identifier uniquely. In XML-based configuration metadata, you use the id and/or name attributes to specify the bean identifier(s).

- **scope**: This attribute specifies the scope of the objects created from a particular bean definition.
- **constructor-arg**: This is used to inject the dependencies.
- **property**: This is used to inject the dependencies.
- **autowire**: This is used to inject the dependencies.
- **lazy-init**: A lazy-initialized bean tells the IoC container to create a bean instance when it is first requested, rather than at startup.
- **init-method**: A callback to be called just after all necessary properties on the bean have been set by the container.
- **destroy-method**: A callback to be used when the container containing the bean is destroyed.

There are three ways to configure the meta-data in Spring FW through a spring configuration file :

- XML Based
- Annotation Based
- Java Based

2.4.9 Spring Bean Factory - Example

Spring 'BeanFactory' Container based 'Hello World' example :

One POJO/Bean : which is injected into an application

An main java class for the application, which has the business logic.

Meta-Data Configuration – XML based, tieing all the java classes to work together

Bean class : HelloWorld.java

```
package com.tcs.ilp.spring.examples;

public class HelloWorld  {
```

```
        private String message;

        public void setMessage(String message){
                this.message  = message;
  }
        public void getMessage(){
    System.out.println("Your Message is: " + message);
  }
}
```

All the Java Bean characteristics need to be followed as per specifications.

In this example, only on member variable, along with setter & getter methods are defined.

Application/Main class : SimpleAppIn.java

```
package com.tcs.ilp.spring.examples;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;

public class SimpleAppIn {
  public static void main(String[] args) {
    XmlBeanFactory factory = new XmlBeanFactory
                    (new ClassPathResource("Beans.xml"));

    HelloWorld obj = (HelloWorld) factory.getBean("helloWorld");
    obj.getMessage();
```

```
  }
}
```

The main or business application uses the beans defined in other part of the application or from some other module. This uses HelloWorld bean class.

The configuration are placed in Beans.xml, which is read by the main class & used as needed by the business logic.

Beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-3.3.xsd">

        <bean id="helloWorld" class=" com.tcs.ilp.spring.examples.HelloWorld">
                <property name="message" value="Hello World!"/>
        </bean>
</beans>
```

Note the configuration :

The beans are defined with id and class name with package name, additionally properties.

Resources like DB connections, JMS connections, etc could also be defined as beans and used as regular beans. Make sure to set the required libs in CLASSPATH

Now run the application to see the output :

Your Message is : Hello World!

When the main SimpleAppln class is executed, it reads the Beans.xml detail and keeps the configuration details in factory object. The container at runtime finds a HelloWorld bean is required by the Simple Appln. The container injects an instance of the bean by automatically creating the bean from HelloWorld class, with the required property set before injection.

2.4.10 Spring Application Context - Example

Implementation Types :

FileSystemXmlApplicationContext: container loads the definitions of the beans from an XML file(with fully resolved path).

ClassPathXmlApplicationContext : container loads the definitions of the beans from an XML file (from CLASSPATH, not with fully resolved file path)

WebXmlApplicationContext: container loads the XML file with definitions of all beans from within a web application.

HelloWorld.java

```
package com.tcs.ilp.spring.examples;

public class HelloWorld  {

        private String message;

        public void setMessage(String message){

                this.message  = message;

  }

        public void getMessage(){

    System.out.println("Your Message is: " + message);

  }

}
```

SimpleAppIn.java

```
package com.tcs.ilp.spring.examples;

import org.springframework.beans.factory.InitializingBean;

import org.springframework.context.ApplicationContext;

import org.springframework.context.support.FileSystemXmlApplicationContext;

public class SimpleAppIn {

  public static void main(String[] args) {

    ApplicationContext context = new FileSystemXmlApplicationContext

                        (“C:\Beans.xml");


    HelloWorld obj = (HelloWorld) factory.getBean("helloWorld");

    obj.getMessage();
```

```
  }
}
```

Beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.3.xsd">

   <bean id="helloWorld" class=" com.tcs.ilp.spring.examples.HelloWorld">
<property name="message" value="Hello World!"/>
   </bean>
</beans>
```

Output : Your Message is : Hello World!

2.4.11 Spring Bean Configuration

Scope	Descriptions
Singleton (default)	Scopes a single bean definition to a single object instance per Spring IoC container.
prototype	Scopes a single bean definition to any number of object instances
Request *	Scopes a single bean definition to the lifecycle of a single HTTP request; that is, each HTTP request has its own instance of a bean created off the back of a single bean definition.
Session *	Scopes a single bean definition to the lifecycle of an HTTP Session.
globalsession *	Scopes a single bean definition to the lifecycle of a global HTTP Session. Typically only valid when used in a Portlet context.

2.4.11.1 Bean Scope :

The scope attribute in the bean definition defines the scope of the objects created. It becomes important & useful when there are multiple objects from a bean definition. Please refer the below table to analyse and know about the various scopes of the bean definition). Note: * indicated scope is applicable only for WebAwareApplicationContext.

Singleton example :

```
<bean id="accountService" class="com.tcs.ilp.spring.example.DefaultAccountService"/>

<!-- the following is equivalent, though redundant (singleton scope is the default) →

<bean id="accountService" class="com.tcs.ilp.spring.example.DefaultAccountService"
scope="singleton"/>
```

Prototype example :

```
<!-- using spring-beans-2.0.dtd -->

<bean id="accountService" class="com.tcs.ilp.spring.example.DefaultAccountService"
scope="prototype"/>
```

Request example (for Web aware ApplicationContext only):

```
<bean id="loginAction" class="com.tcs.ilp.spring.example.LoginAction" scope="request"/>
```

Session example (for Web aware ApplicationContext only):

```
<bean id="userPreferences" class="com.tcs.ilp.spring.example.UserPreferences"
scope="session"/>
```

Global Session example (for Web aware ApplicationContext only):

```
<bean id="userPreferences" class="com.tcs.ilp.spring.example.UserPreferences"
scope="globalSession"/>
```

2.4.11.2 Bean Inheritance:

Bean definitions can be inherited like the Java class inheritance, with parent attribute.

‘abstract’ attribute based inheritance is also allowed.

Bean Inheritance – Example:

Bean definition hierarchy :

```
<bean id="inheritedTestBean" abstract="true” class="org.springframework.beans.TestBean">
        <property name="name" value="parent"/>
        <property name="age" value="1"/>
</bean>
<bean id="inheritsWithDifferentClass” class="org.springframework.beans.DerivedTestBean"
                parent="inheritedTestBean" init-method="initialize">
        <property name="name" value="override"/>
        <!-- the age property value of 1 will be inherited from parent -->
</bean>
```

Bean Inheritance – Example:

Abstract Bean definition hierarchy example :

```
<bean id="inheritedTestBeanWithoutClass" abstract="true">
        <property name="name" value="parent"/>
        <property name="age" value="1"/>
</bean>
<bean id="inheritsWithClass" class="org.springframework.beans.DerivedTestBean"
                parent="inheritedTestBeanWithoutClass" init-method="initialize">
```

```
        <property name="name" value="override"/>

        <!-- age will inherit the value of 1 from the parent bean definition-->

</bean>
```

2.4.12 Spring Bean Life cycle

The IoC Container manages all the beans from instantiation till destruction & it does lots of activities between these period without users awareness. During instantiation, there are some Initializations done to make the object usable. And during destruction of object, some resource cleanup is required. For these two important activities, the Spring FW provides Two Callback

Initialization Callback	Destruction Callback
org.springframework.beans.factory.InitializingBean	org.springframework.beans.factory.DisposableBean
void afterPropertiesSet() throws Exception;	void destroy() throws Exception;
init-method attribute in bean definition	destroy-method

public class ExampleBean implements InitializingBean {
public void afterPropertiesSet() {
// do some initialization work
}
}
	public class ExampleBean implements DisposableBean {
	public void destroy() {
	// do some destruction work
	}
	}

Methods** so that container does the above mentioned objects management. The developer/designers can be make use of in their application development activities :

afterPropertiesSet() provided in InitializingBean callback interface

destroy() provided in DisposableBean callback interface

Definition of Call Back Methods : a callback is a piece of executable code that is passed as an argument to other code, which is expected to call back (execute).

2.4.13 Spring Container Auto-wiring

In some applications, there might be lot of Beans to handle within the scope. That means a lots of Configuration, running into multiple pages, that obviously brings a huge complexity in maintaining the configuration itself.

The Spring FW's Auto-wiring will help such situations; It can auto-wire dependencies through introspection of the bean classes so that you do not have to explicitly specify the bean properties or constructor arguments.

'auto-wire' is the keyword to configure the option, to be defined in the bean definition.

```
<bean id="BEAN-NAME" class=" CLASS-NAME-W-PKG" autowire="MODE">
```

Auto-wiring has its own advantages and disadvantages. Depending on the situation in consideration, it should be chosen for implementation.

Advantages :

- Auto-wiring can significantly reduce the volume of configuration required.
- Auto-wiring can cause configuration to keep itself up to date as your objects evolve.
- Auto-wire reduces or eliminates the need to specify properties or constructor arguments, thus saving a significant amount of typing.

Disadvantages :

- Auto-wiring is more magical than explicit wiring. Although, Spring FW is careful to avoid guessing in case of ambiguity which might have unexpected results, the relationships between your Spring-managed objects are no longer documented explicitly.
- Wiring information may not be available to tools that may generate documentation from a Spring container.